Practice midterm 1 (with solutions)

TC1037 Algorithmic Thinking · Group 510 · Tecnológico de Monterrey, Campus Querétaro

Practice — download, solve on paper in 80 minutes, then check the solutions

Name: _____ **Matrícula:** _____

80 minutes · 100 points · individual · on paper. No computer, phone, notes or calculator. Write in pen or dark pencil; code must be legible and indented. **Part I** (30 pts): write *exactly* what Python prints — every line, in order. **Part II** (70 pts): write the program. Suggested time: 15 minutes for Part I, 65 for Part II. **This is a practice paper: same format, same difficulty, different problems.**

Question:	1	2	3	4	5	6	7	8	9	10	11	Total
Points:	5	5	5	5	5	5	23	23	3	20	1	100
Score:												

Part I — Read and predict

(30 points)

1. **Arithmetic operators.** What does Python print? [5]

```
print(6 + 9 // 4 * 3 - 7 % 2 ** 3)
```

Output:

2. **Boolean operators.** What does Python print? [5]

```
print(not 2 >= 2 or 6 / 3 == 2 and 1 < 0)
```

Output:

3. **Conditional structure.** What does Python print? [5]

```
x = 20
y = 6
if x % y == 2:
    print("A", x // y)
elif x > 10:
    print("B", x % y)
else:
    print("C")
print("end")
```

Output:

4. **Functions.** What does Python print? [5]

```python
def h(x, y):
    x = x + y
    y = x - y
    return x * y

def main():
    x = 2
    y = 3
    z = h(y, x)
    print(x, y, z)

main()
```

Output:

5. **The while loop.** What does Python print? [5]

```python
n = 50
count = 0
while n > 5:
    n = n // 2
    count = count + 1
print(n, count)
```

Output:

6. **The for loop.** What does Python print? [5]

```python
for i in range(1, 12, 4):
    print(i * i)
```

Output:

Part II — Write the program **(70 points)**

Write **one** Python program with the following parts. All `input()` and all validation happen in `main()`; the two functions **return** their result and never print or read. Write each function's purpose in a one-line comment above it, and under each loop a one-line comment saying why it stops (`while`) or how many times it runs (`for`).

7. The function `sum_digits(n)` receives a non-negative integer `n` and **returns** the sum of its digits. Use a `while` loop with `%` and `//`. [23]

   Examples: `sum_digits(1234)` → `10` `sum_digits(9)` → `9` `sum_digits(1000)` → `1` `sum_digits(0)` → `0`

8. The function `count_multiples(n, k)` receives two integers $n \geq 1$ and $k \geq 1$ and **returns** how many multiples of k there are from 1 to n, inclusive. Use a `for` loop. [23]

   Examples: `count_multiples(10, 3)` → `3` `count_multiples(12, 3)` → `4` `count_multiples(5, 7)` → `0` `count_multiples(7, 7)` → `1`

9. The function `menu()` that displays: `1. sum_digits 2. count_multiples 3. Exit` [3]

10. The function `main()` that, inside a `while` loop: shows the menu, reads the option, and [20]
    - option 1: reads n (must be 0 or more); if any value is out of range displays a clear message, otherwise calls `sum_digits` and displays the result with a label;
    - option 2: reads n and k (both must be 1 or more); same rule, calling `count_multiples`;
    - option 3: displays `Goodbye.` and the program ends;
    - any other option: displays `Invalid option.` and the program ends.

    Use `if`/`elif`/`else` to choose the option.

11. The last line of the script calls `main()`. [1]

Technical requirements — 2 points off for each one missing: meaningful variable names in Python style · a clear prompt for every input · readable, labelled output.

Part II — your program (continue on the back if needed)

How Part II is graded (the academia's rubric)

Each part of the program (the two functions, `menu`, `main`, the call) is scored on this scale, as a percentage of its points:

Excelente 100%	**Muy bien 85%**	**Regular 70%**	**Deficiente 50%**	**Muy deficiente 20%**	**Sin evidencia 0%**
No syntax errors; works correctly, including the boundary cases in the examples.	Correct approach; minimal logic or syntax slips.	At most two syntax errors and/or small logic errors; the approach is closely related to what was asked.	Approach loosely related, or related but with more than two syntax errors.	Code unrelated to the problem.	No code.

Part I: 5 points per question, all or nothing — every line of the output, in order. Technical requirements: -2 each, from Part II.